1

solution - 1

description

- Generate an HTTP-compliant request string and send it to the server to make a socket-based HTTP request.
- Since the server's port number is 443 (which uses the SSL layer), use `ssl.wrap_socket()` for transmission.
- Separate the server's response into header and body, then parse the body as JSON.

exception handle

- Handle exceptions for potential errors that may occur when parsing the server's response.

screenshot

```
● (np-hw-1) ➜  np-hw-1 git:(main) ✗ python solution1.py
  /Users/moodtrbl/PythonProjects/np-hw-1/solution1.py:22: DeprecationWarning: ssl.wrap_socket() is deprecated, use SSLContext.wrap_socket()
    ssl_socket = ssl.wrap_socket(sock)
  local_time = 2026-03-29T16:06:42.093326+09:00
  timezone   = Asia/Seoul
  utc_time   = 2026-03-29T07:06:42.093326+00:00
○ (np-hw-1) ➜  np-hw-1 git:(main) ✗
```

solution - 2

server

description

- Since UDP is used, use `sock.recvfrom()` to receive data.
- When counting vowels, store the discovered vowels in a list to use for subsequent output.
- Send the count of discovered vowels back to the client.
- Use a `while` loop to continue receiving requests even after a client connection ends.

screenshot

```
○ (np-hw-1) ➜  np-hw-1 git:(main) ✗ python solution2_server.py
  Vowels found : e, o, o, a, i, i, u → 7 vowels
  Vowels found : e, o → 2 vowels
  Vowels found : i, e, e, A, a, E, i, O, u → 9 vowels
```

client

description

- Send the input string to the server.
- Display the count received in the response.

exception handle

- Set a timeout to handle exceptions when the server does not respond.

screenshot

```
Server reply : Vowel count: 1
(np-hw-1) → np-hw-1 git:(main) ✗ python solution2_client.py
Client sends : Network programming is fun
Server reply : Vowel count: 7
(np-hw-1) → np-hw-1 git:(main) ✗ python solution2_client.py
Client sends : hello
Server reply : Vowel count: 2
(np-hw-1) → np-hw-1 git:(main) ✗ python solution2_client.py
Client sends : is there AnY aEiOu?
Server reply : Vowel count: 9
```

solution - 3

server

description

- Start the game only if the client sends 'start'; otherwise, return "Error 400: Bad Request. Send 'start' to play."
- Generate a problem using `random.randint(1, 20)`.
- If the answer is correct, terminate the loop and return "Correct! You win."
- If the answer is incorrect, return "Incorrect. Try again!"
- If the number of attempts exceeds `MAX_ATTEMPTS`, return "Game Over. Out of attempts. The correct answer was [n]."
- Use a `while` loop to continue receiving requests even after a client connection ends

exception handle

- Attempt to cast the input to an `int`, and handle exceptions if the client does not send an integer answer.

client

description

- Send "start" to initiate the game.
- Run a loop to allow up to `MAX_ATTEMPTS`.
- If the response indicates a correct answer, end the game; if incorrect, increment the attempt count; if the response is "Game Over," terminate the game.
- Send the value input by the user to the server.
- Before sending the value to the server, use `get_integer_answer()` to verify if it is an integer.

exception handle

- Handle exceptions for potential connection failures when attempting `connect()`.
- In `get_integer_answer()`, perform an `int` cast to check for integer type; if an error occurs, determine it is not an integer and handle the exception.

screenshot

```
● (np-hw-1) ➜  np-hw-1 git:(main) ✗ python solution3_client.py
  What is 17 + 18?
  Attempt: 1
  hello
  Invalid value. input int
  Attempt: 1
  123
  Incorrect. Try again!
  Attempt: 2
  123
  Incorrect. Try again!
  Attempt: 3
  123
  Game Over. Out of attempts. The correct answer was [35].
```

```
● (np-hw-1) ➜  np-hw-1 git:(main) ✗ python solution3_client.py
  What is 16 + 2?
  Attempt: 1
  -1
  Incorrect. Try again!
  Attempt: 2
  3
  Incorrect. Try again!
  Attempt: 3
  18
  Correct! You win.
```